

Projeto e Análise de Algoritmos

Busca em Grafos

Thiago Pinheiro de Araújo

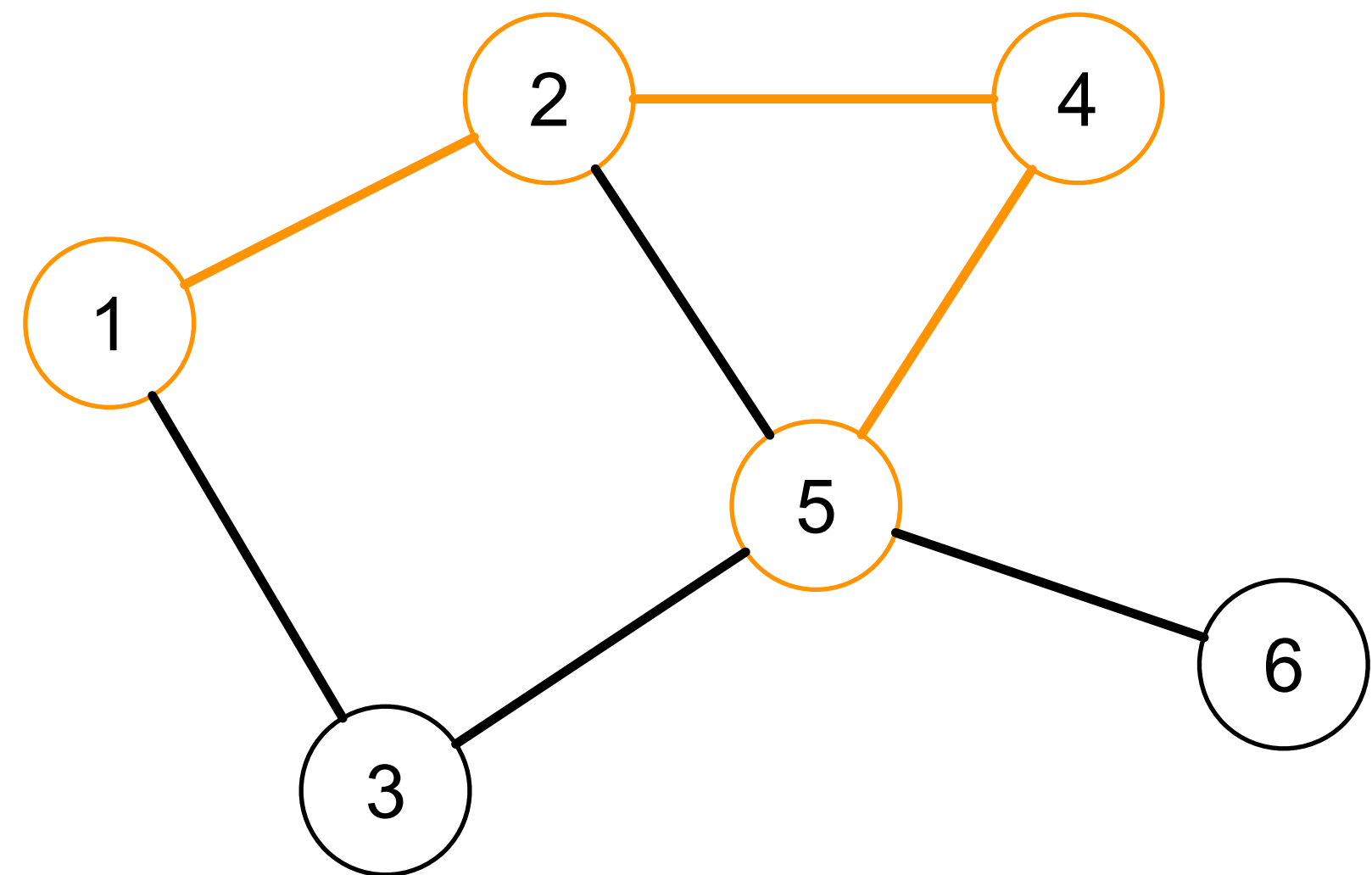
EMAp/FGV

2025.2

Discutir **caminhos e conectividade** em grafos e apresentar as principais estratégias utilizadas para construir algoritmos de busca.

Caminhos

- Um **caminho** P em $G = (V, E)$ consiste em uma sequência com n vértices, finita e não vazia, tal que v_{i+1} é adjacente de v_i para todo $i < n$.
- O caminho é simple se não possuir vértices repetidos.
- O caminho é fechado (ciclo) se $v_1 = v_n$.
- O comprimento de um caminho é definido pelo número de arestas presentes nele
 - Um caminho que passa por n vértices possui $n - 1$ arestas.



Caminhos

- Dado um grafo $G = (V, E)$ e um caminho P composto por uma sequência de vértices podemos verificar se P é um caminho de G .

```
bool isPath(const vertex path[], int lenght) {  
    if (lenght < 2) { return false; } // The path must have at least 2 vertices  
  
    // Get the first vertex  
    int vertexInx = 0;  
    vertex v1 = path[vertexInx++];  
  
    // Check if there is an edge for each pair of vertices  
    do {  
        vertex v2 = path[vertexInx++];  
        if (!hasEdge(v1, v2)) { // If there is no edge, it is not a path  
            return false;  
        }  
        v1 = v2;  
    } while (vertexInx < lenght);  
    return true;  
}
```

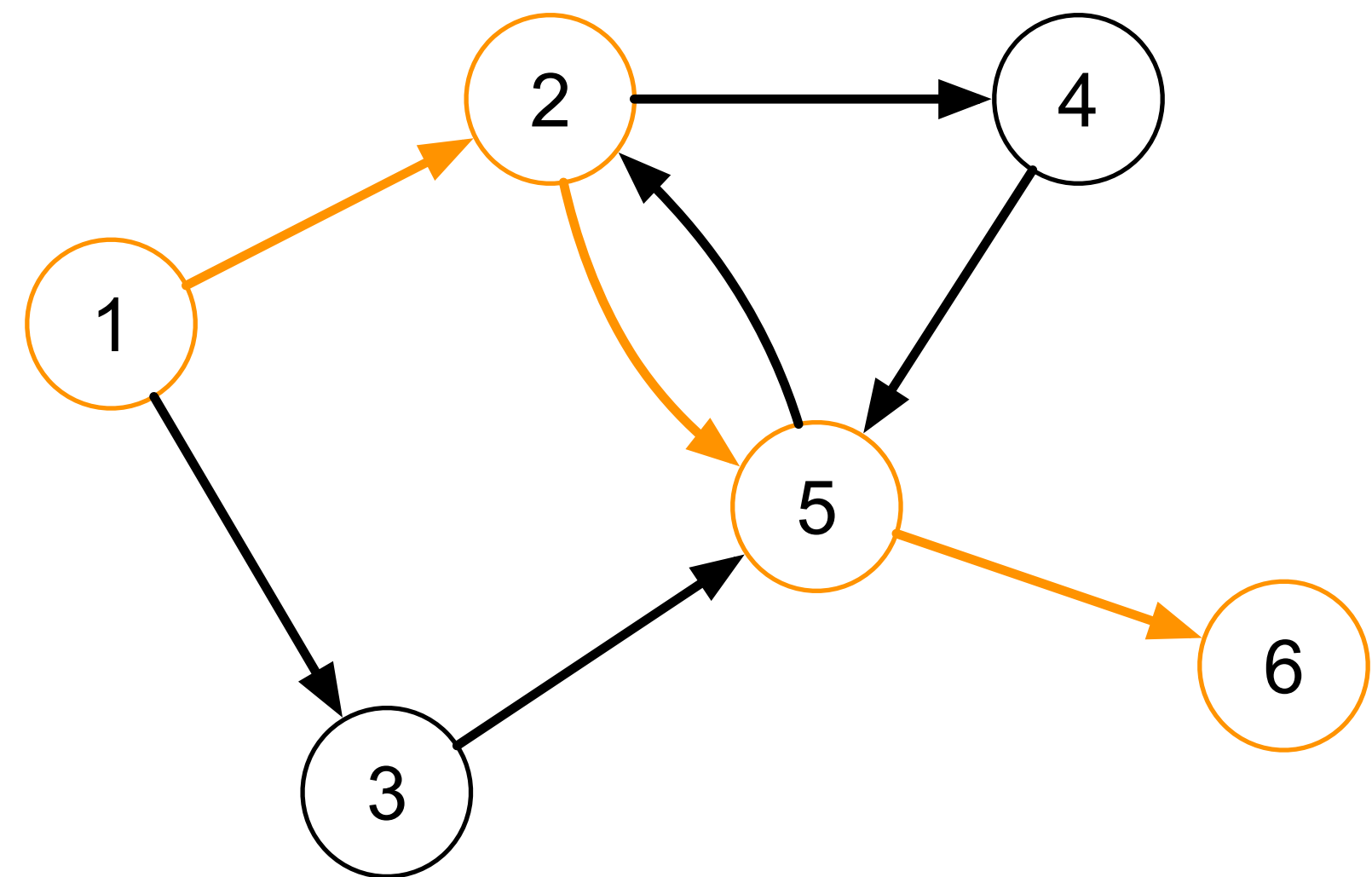
Caminhos

- Como verificar se o caminho é simples?
- Qual é a complexidade de tempo?
 - Depende da estrutura de dados utilizada para representar o grafo!

```
bool isPath(const vertex path[], int lenght, bool & hasCycle) {  
    if (lenght < 2) { return false; } // The path must have at least 2  
    vertices  
  
    // Get the first vertex  
    int vertexInx = 0;  
    vertex v1 = path[vertexInx++];  
  
    // Create an array to mark as visited  
    int visited[m_numVertices];  
    for (vertex v=0; v < m_numVertices; v++) { visited[v] = false; }  
    visited[v1] = true;  
  
    // Check if there is an edge for each pair of vertices  
    do {  
        vertex v2 = path[vertexInx++];  
        if (!hasEdge(v1, v2)) { // If there is no edge, it is not a path  
            return false;  
        }  
        if (visited[v2]) { // If was visited, there is a cycle  
            hasCycle = true;  
        }  
        visited[v2] = true;  
        v1 = v2;  
    } while (vertexInx < lenght);  
    return true;  
}
```

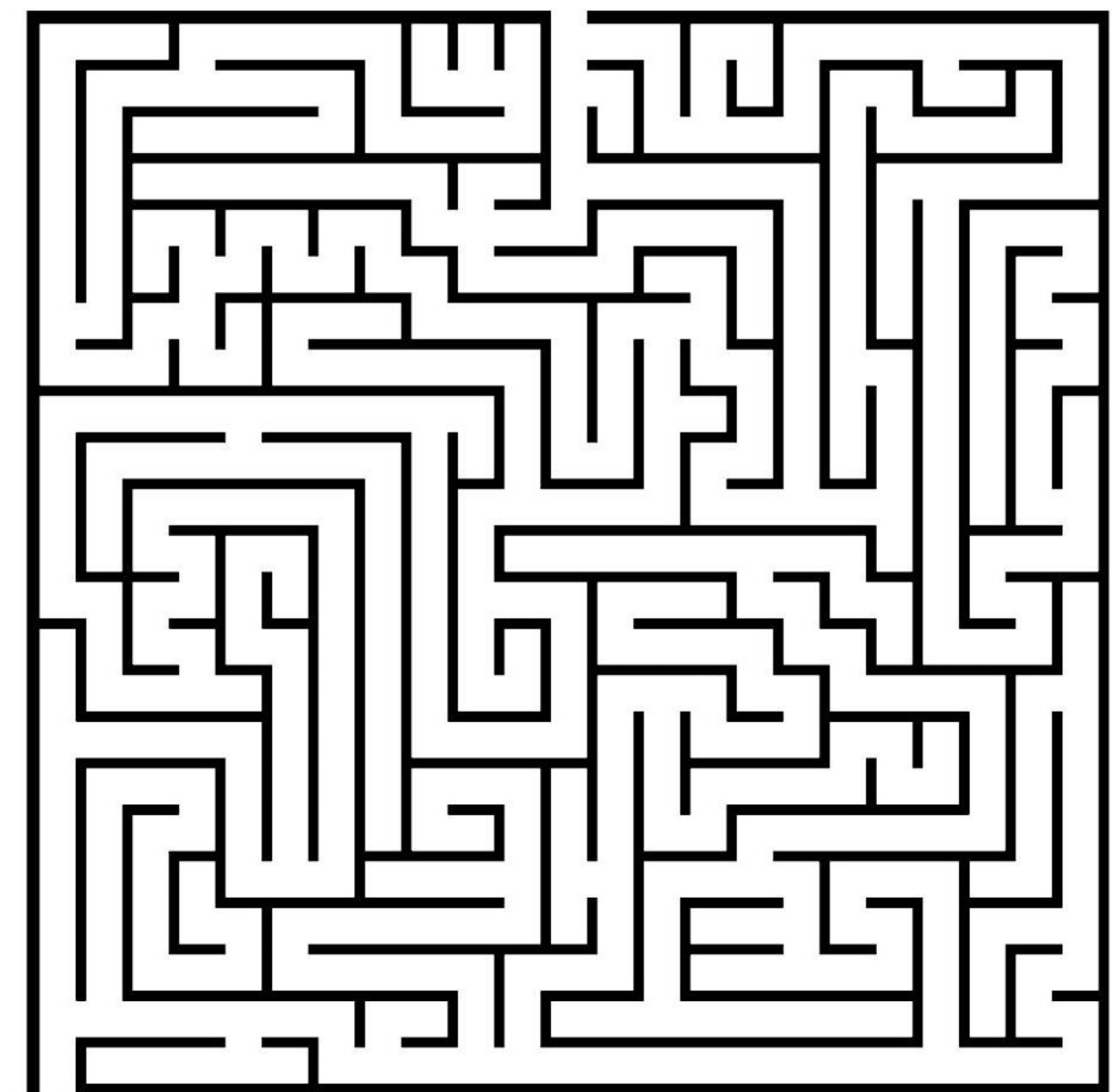

Acessibilidade

- **Problema:** dado um par de vértices v_i e v_j verificar v_j pode ser alcançado iniciando um caminho em v_i .
- A figura ao lado mostra o caminho:
 - $P = \{1, 2, 5, 6\}$
- Existe um caminho de v_4 até v_3 ?



Acessibilidade

- A solução geral é baseada no método de exploração de labirintos (Trémaux, século XIX).
- Como fazer para não se perder no labirinto (grafo) e percorrer cada segmento (aresta) uma única vez?
- Durante um passeio pelo labirinto (busca) cada vez que encontrar uma junção (vértice) marca que ele foi visitado.
- Volte se encontrar uma junção visitada ou um beco sem saída (sorvedouro).



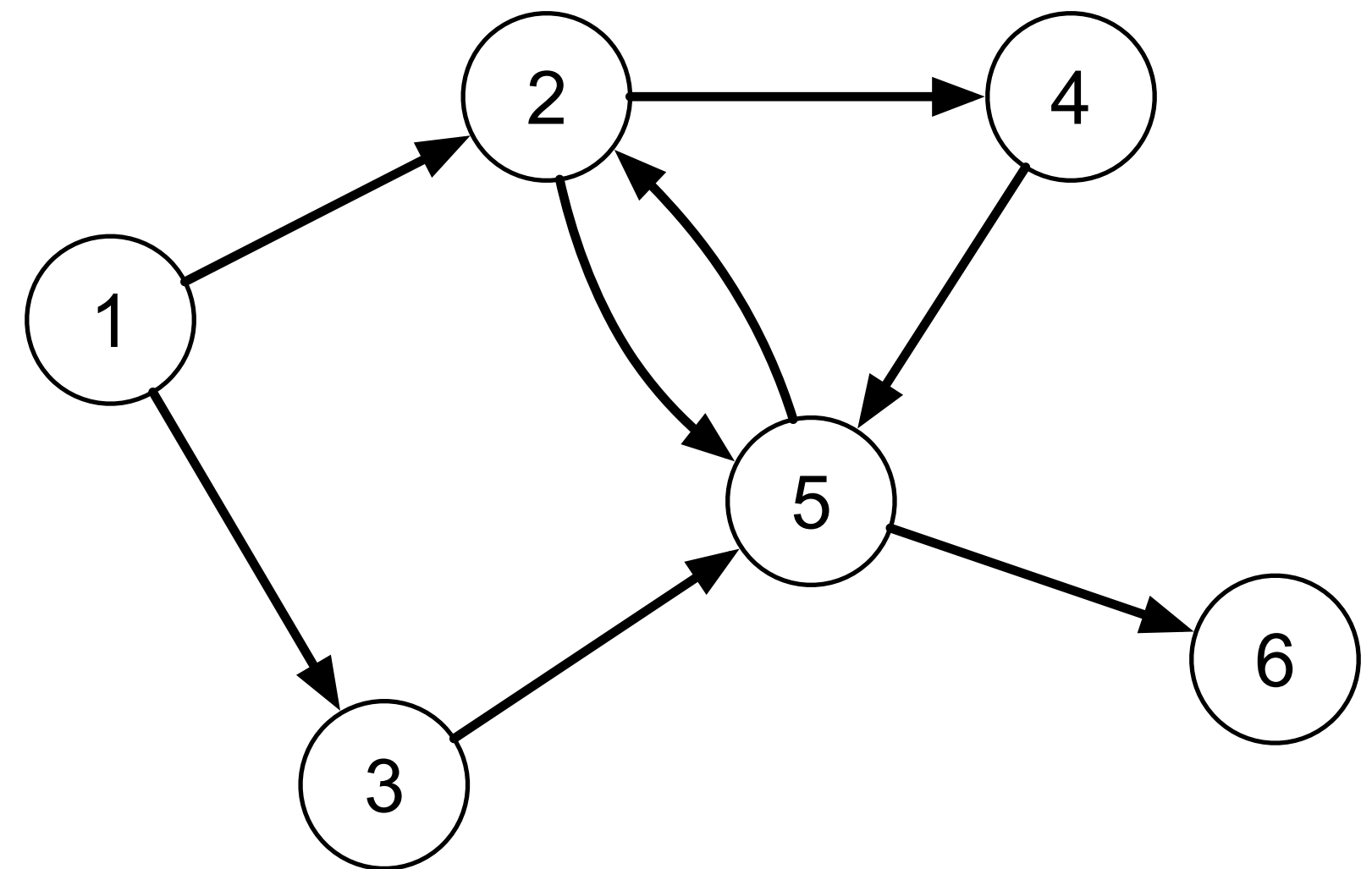
- **Voltando ao problema:** podemos escrever um algoritmo que percorre o grafo a partir do vértice v_i armazenando os vértices visitados; e, ao final, verifica se v_j foi visitado.

```
bool canReach(vertex v1, vertex v2) {  
    bool visited[m_numVertices];  
    for (vertex v=0; v < m_numVertices; v++) { visited[v] = false; }  
    reachRecursive(v1, visited);  
    return visited[v2];  
}
```

```
void reachRecursive(vertex v1, bool * visited) {  
    visited[v1] = true;  
    for (vertex v2 = 0; v2 < m_numVertices; v2++) {  
        if (hasEdge(v1, v2) && !visited[v2]) {  
            reachRecursive(v2, visited);  
        }  
    }  
}
```


- Como seria a execução desse algoritmo no grafo a seguir, buscando um caminho entre v_1 e v_6 ?

$(1, 2)$	100000
$(2, 4)$	110000
$(4, 5)$	110100
$(5, 2)$	110110
$(5, 6)$	110110
$(2, 5)$	110111
$(1, 3)$	110111
$(3, 5)$	111111



Acessibilidade

- Observe que o resultado também indica todos os demais vértices que podem ser alcançados a partir de v_i
- Se a acessibilidade fosse o único objetivo a execução poderia ter sido interrompida ao encontrar o vértice v_j .

Busca em profundidade

- Um **algoritmo de busca** (ou varredura) em grafo é qualquer algoritmo que visita todos os vértices percorrendo as arestas definidas
 - A ordem de pesquisa depende do algoritmo.
- O algoritmo de busca em profundidade (DFS - *depth-first search*) é uma versão mais completa do algoritmo anterior.
- É uma estrutura de solução genérica utilizada em diversos problemas específicos.

Busca em profundidade

- O algoritmo de **busca em profundidade (DFS)** consiste em visitar todos os vértices ao menos uma vez e levantar propriedades sobre a estrutura do grafo
 - Vamos começar identificando a ordem de descoberta dos vértices (ou pré-ordem).

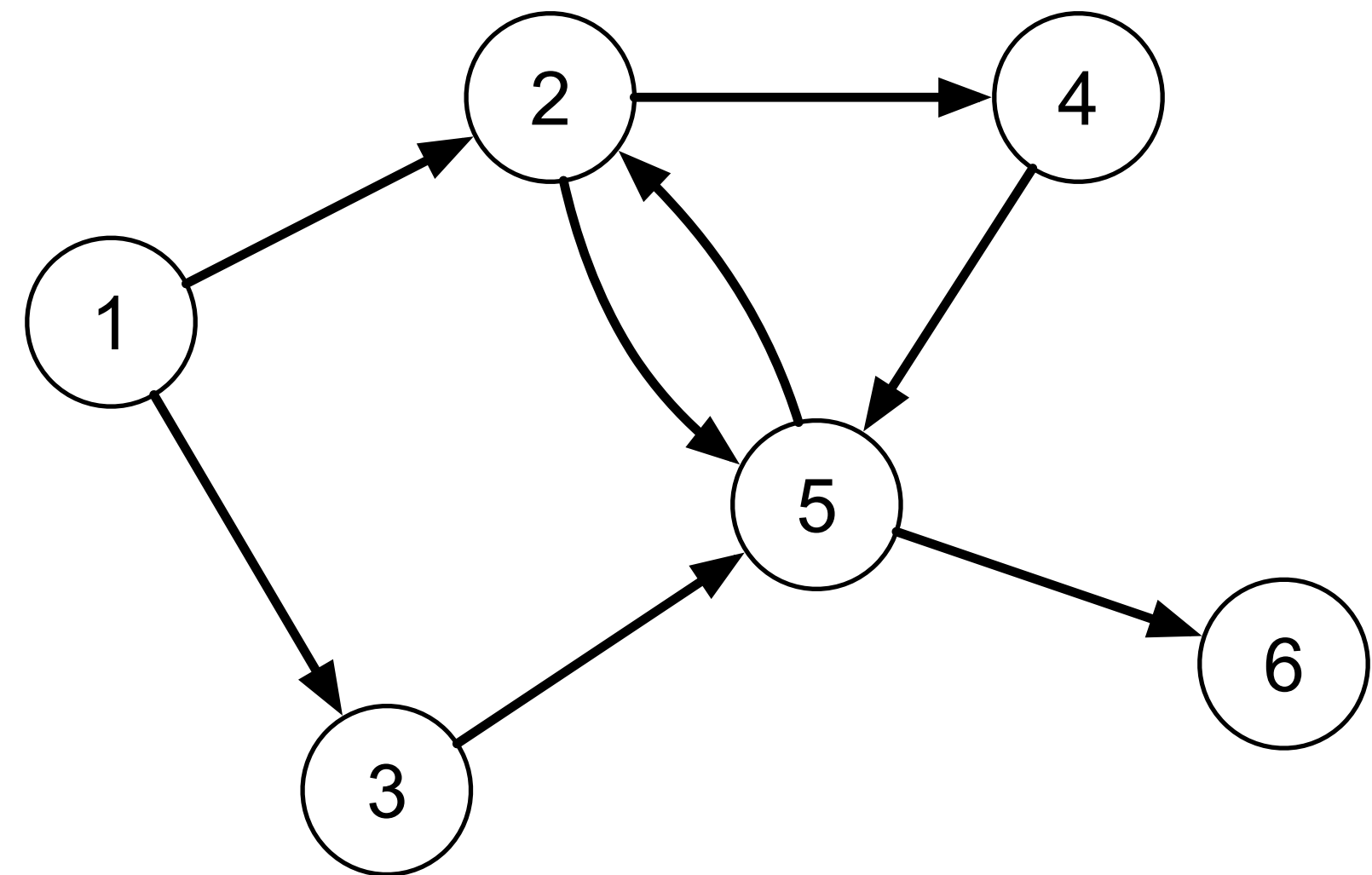
```
void dfs(int * preOrder) {
    int counter = 0;
    for (vertex v=0; v < m_numVertices; v++) {
        preOrder[v] = -1;
    }
    for (vertex v=0; v < m_numVertices; v++) {
        if (preOrder[v] == -1) {
            dfsRecursive(v, preOrder, counter);
        }
    }
}
```

```
void dfsRecursive(vertex v1, int * preOrder,
                  int & counter) {
    preOrder[v1] = counter++;
    EdgeNode * edge = m_edges[v1];
    while(edge) {
        vertex v2 = edge->otherVertex();
        if (preOrder[v2] == -1) {
            dfsRecursive(v2, preOrder, counter);
        }
        edge = edge->next();
    }
}
```

- Como seria a execução desse algoritmo no grafo a seguir?

(1, 3)	0	–	–	–	–	–
. (3, 5)	0	–	1	–	–	–
.. (5, 2)	0	–	1	–	2	–
... (2, 5)	0	3	1	–	2	–
... (2, 4)	0	3	1	–	2	–
.... (4, 5)	0	3	1	4	2	–
.. (5, 6)	0	3	1	4	2	–
(1, 2)	0	3	1	4	2	5

v	=	1	2	3	4	5	6
<i>preOrder</i> [v]	=	0	3	1	4	2	5



- Dizemos que um vértice v é visitado quando $preOrder[v]$ é consultado.
- Dizemos que um vértice v é descoberto quando $preOrder[v]$ é definido.
- A busca poderia ser iniciada em qualquer vértice
 - A implementação de exemplo inicia em zero apenas por comodidade.

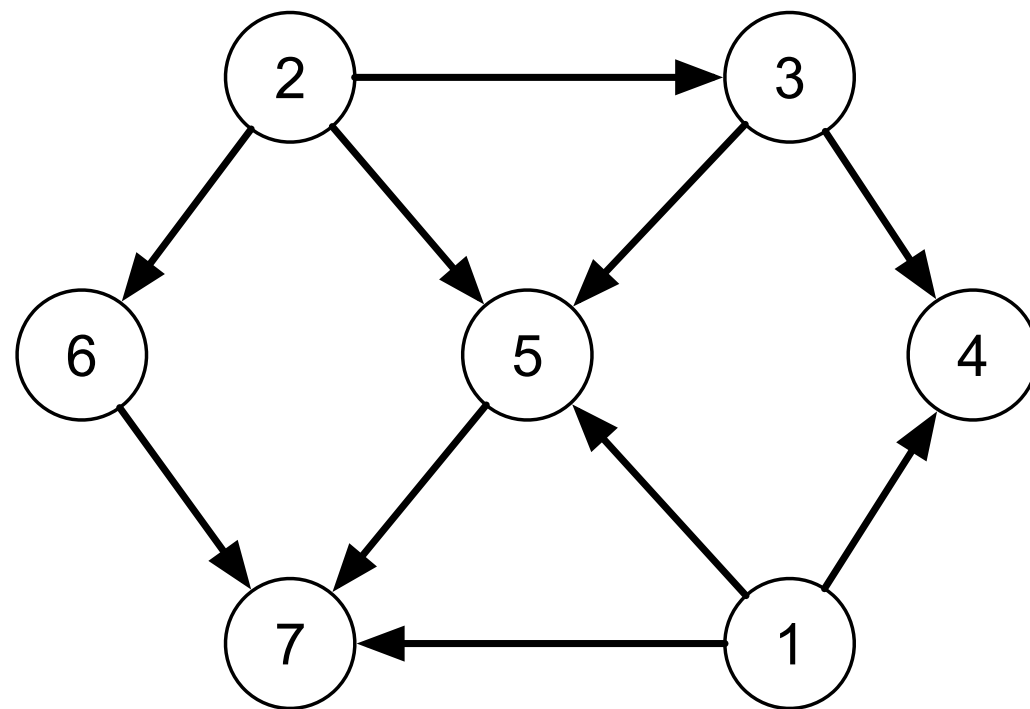
Busca em profundidade

- A abordagem de tentar percorrer a partir de cada vértice garante que todos serão inspecionadas
 - Mesmo que o grafo não seja conexo.
 - Essa é a principal diferença com relação ao algoritmo de detecção de caminho.
- Um grafo pode apresentar múltiplas sequências de pré-ordem
 - Depende da ordem em que as arestas são inspecionadas.

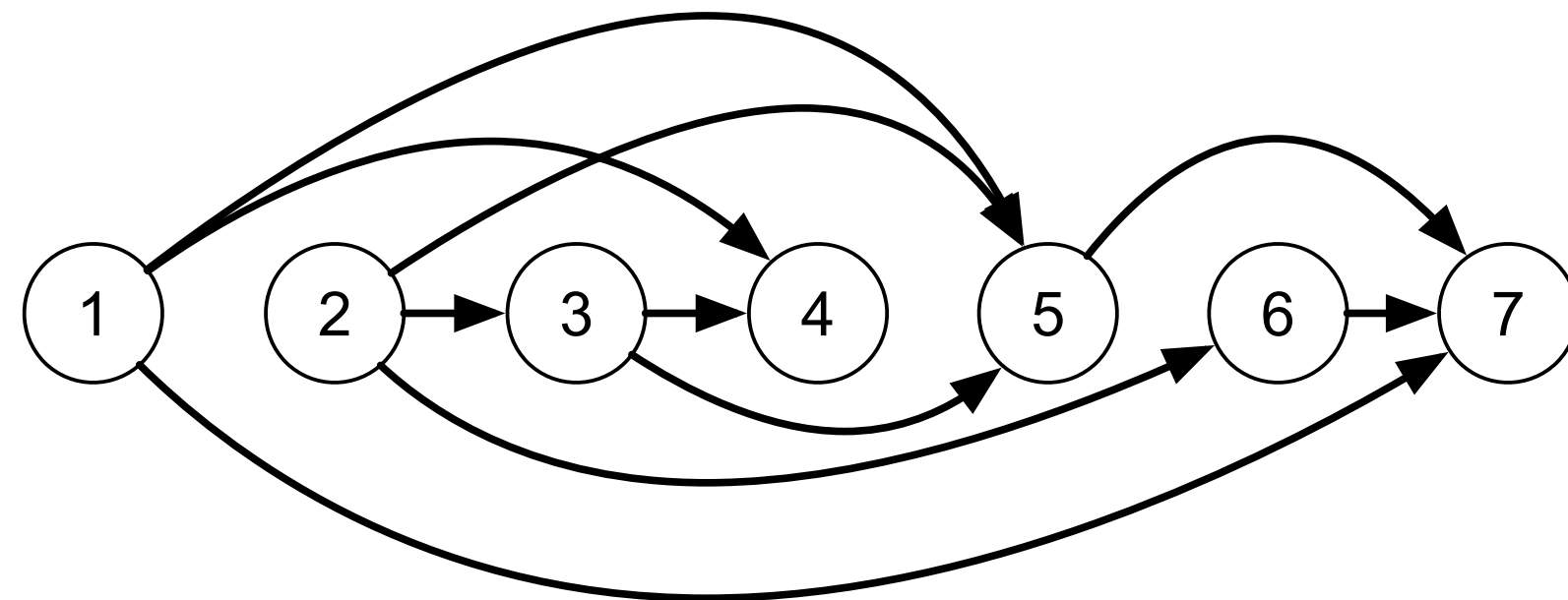
- Analise da complexidade:
 - Cada vértice é processado uma única vez.
 - Em cada vértice v_i são verificados $g_s(v_i)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $\Theta(V + E)$ ao utilizar a estrutura de dados lista de adjacências
 - Se a estrutura de dados for matriz de adjacências a complexidade vai ser $\Theta(V^2)$.
- Se o grafo for denso, em ambas as estruturas de dados a complexidade será $\Theta(V^2)$.

Grafos topológicos

- Um **grafo topológico** é um grafo que admite uma ordenação dos vértices de forma que para toda aresta (v_i, v_j) temos que $i < j$
 - Observe que se os vértices forem dispostos em ordem crescente em uma linha toda aresta irá apontar para o sentido de crescimento dos números.



É topológico?



Grafos topológicos

- Grafos topológicos não apresentam ciclos.
- Todo vértice de um grafo topológico é:
 - O término de um caminho que começa numa fonte.
 - A origem de um caminho que termina num sorvedouro.
- Um determinado grafo pode não possuir uma numeração topológica
 - Ou possuir várias diferentes.
- Exemplo de aplicação: pipeline de tarefas.

- **Exercício:** crie um algoritmo que verifica se a numeração dos vértices de um grafo $G = (V, E)$ é topológica.

```
bool isTopological() {  
    for (vertex i=0; i < m_numVertices; i++) {  
        EdgeNode * e = m_edges[i];  
        while (e) {  
            if (i >= e->otherVertex()) {  
                return false;  
            }  
            e = e->next();  
        }  
    }  
    return true;  
}
```

Grafos topológicos

- Como descobrir se um grafo $G = (V, E)$ possui numeração topológica e determiná-la?
- Podemos eliminar uma fonte ($g_e(v_k) = 0$) de G produzindo um subgrafo G' , e repetindo o procedimento sobre ele.
- Numere os vértices removidos.
- Se todos os vértices forem removidos, a numeração é topológica.
- Se restarem vértices, o grafo não possui uma numeração topológica.

- **Exercício:** crie um algoritmo para determinar se um grafo possui ordenação topológica e determiná-la.

```
bool hasTopologicalOrderSlow(int order[]) {
    for (vertex i=0; i < m_numVertices; i++) { order[i] = -1; }
    int counter = 0;
    while (counter < m_numVertices) {
        vertex i = 0;
        for (; i < m_numVertices; i++)
            if (inDegree(i) == 0 && order[i] == -1) { break; }
        if (i >= m_numVertices) { return false; }
        order[i] = counter++;
        EdgeNode * e = m_edges[i];
        while (e) {
            EdgeNode * next = e->next();
            removeEdge(i, e->otherVertex());
            e = next;
        }
    }
    return true;
}
```

Problemas:

- **É lento!**
 - $O(V^2(V + E))$ utilizando lista de adjacências.
- Corrompe a configuração inicial do grafo.

- Uma solução mais eficiente seria:

```
bool hasTopologicalOrder(int order[]) {
    int inDegree[m_numVertices];
    for (vertex i = 0; i < m_numVertices; i++) {
        inDegree[i] = 0;
    }
    for (vertex i = 0; i < m_numVertices; i++) {
        for (EdgeNode * e = m_edges[i];
             e != nullptr; e = e->next()) {
            inDegree[e->otherVertex()]++;
        }
    }
    vertex queue[m_numVertices];
    int start = 0;
    int end = 0;
    for (vertex i = 0; i < m_numVertices; i++) {
        if (inDegree[i] == 0) {
            queue[end++] = i;
        }
    }
}
```

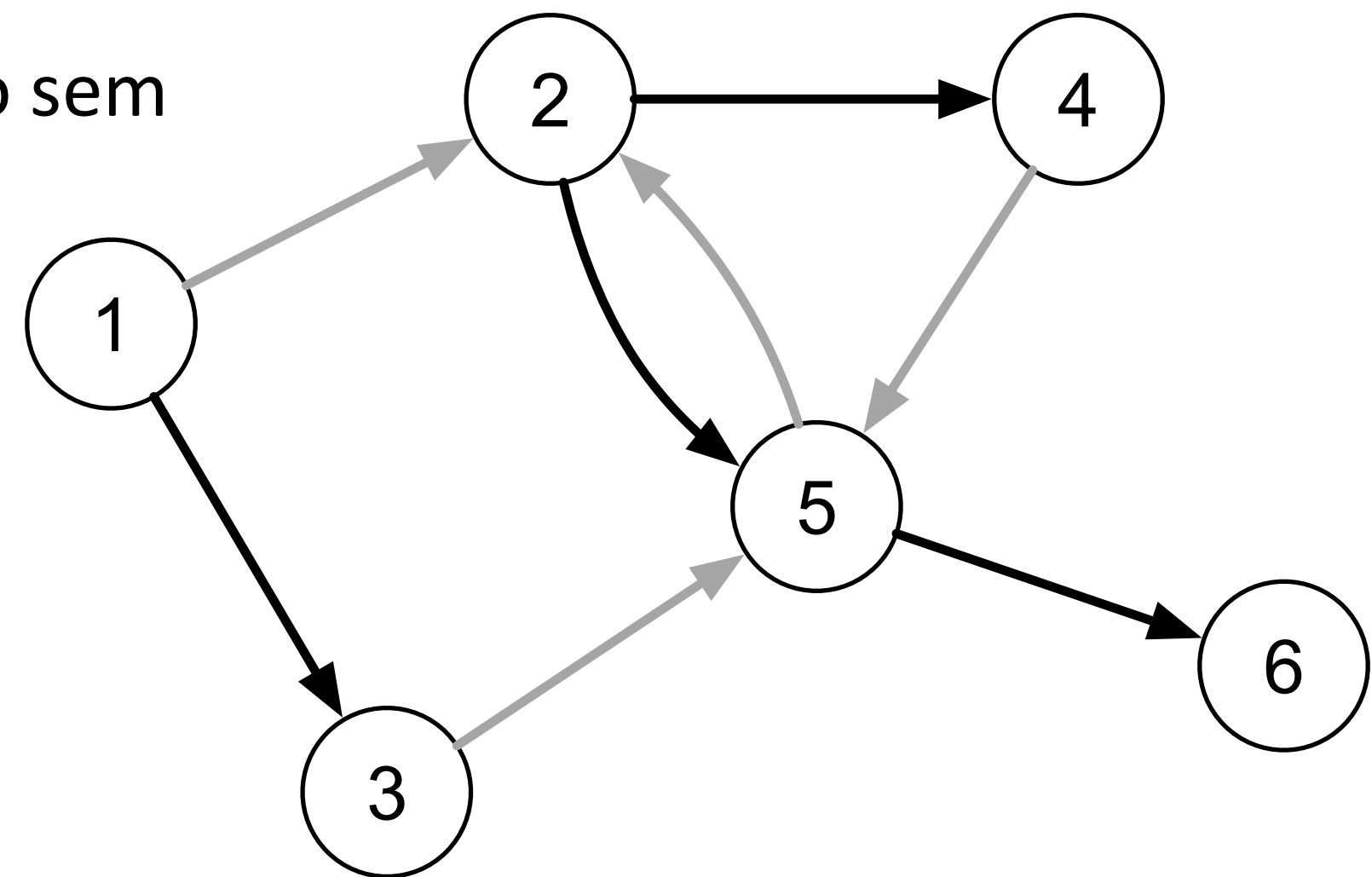
```
int counter = 0;
while (start < end) {
    vertex v = queue[start++];
    order[v] = counter++;
    for (EdgeNode * e = m_edges[v];
         e != nullptr; e = e->next()) {
        vertex other = e->otherVertex();
        inDegree[other]--;
        if (inDegree[other] == 0) {
            queue[end++] = other;
        }
    }
}
return counter >= m_numVertices;
}
```

Complexidade: $O(V + E)$

- Porque esse algoritmo funciona?
- Um determinado vértice v_j somente é numerado após todos que possuem uma aresta (v_i, v_j) serem removidos, consequentemente numerados mantendo $i < j$.
- Todo grafo topológico possui ao menos uma fonte
 - Se o algoritmo terminar antes de remover todos os vértices, ele não encontrou uma fonte
 - Portanto não possui uma numeração topológica.

Florestas DFS

- Observe que o caminho percorrido pelo algoritmo DFS produz um conjunto de florestas radicadas.
- Uma **floresta radicada** é um grafo topológico sem vértices com grau de entrada maior que 1.
- As fontes de uma floresta radicada são as raízes das árvores.
- Os sorvedouros são as folhas das árvores.



**Iniciando no vértice v_2 propositalmente.*

- A floresta gerada pela execução do algoritmo de busca em profundidade também é chamada de **floresta DFS**
 - Essa floresta também é um grafo gerador.
- Todo vértice de uma floresta DFS é término de um caminho que começa em uma fonte.
- Dado que o grau de entrada de cada vértice é no máximo 1, podemos representar a floresta DFS como um vetor de pais (*parents*).

- O algoritmo de **busca em profundidade (DFS)** pode ser modificado para gerar a árvore DFS da seguinte forma:

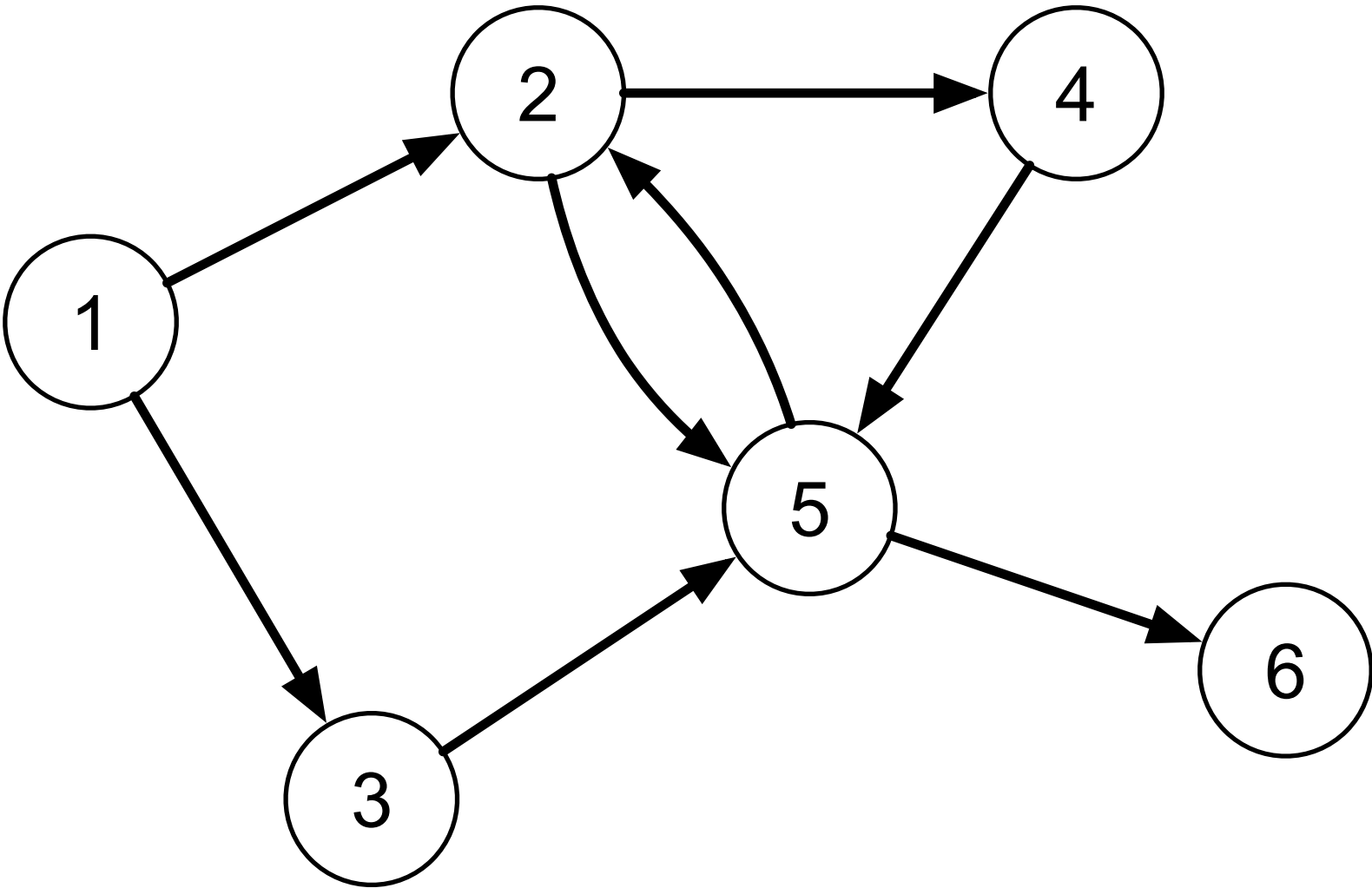
```
void dfs(int * preOrder, int * parents) {
    int counter = 0;
    for (vertex v=0; v < m_numVertices; v++) {
        preOrder[v] = -1;
        parents[v] = -1;
    }
    for (vertex v=0; v < m_numVertices; v++) {
        if (preOrder[v] == -1) {
            parents[v] = v; // Set root
            dfsRecursive(v, preOrder,
                        counter, parents);
        }
    }
}
```

```
void dfsRecursive(vertex v1, int * preOrder,
                  int & counter, int * parents,
                  int level=0) {
    preOrder[v1] = counter++;
    EdgeNode * edge = m_edges[v1];
    while(edge) {
        vertex v2 = edge->otherVertex();
        if (preOrder[v2] == -1) {
            parents[v2] = v1; // Set parent first
            dfsRecursive(v2, preOrder, counter,
                        parents, level + 1);
        }
        edge = edge->next();
    }
}
```

- Como seria a execução desse algoritmo no grafo a seguir?

(1, 3)	0	—	—	—	—	—
. (3, 5)	0	—	0	—	—	—
.. (5, 2)	0	—	0	—	2	—
... (2, 5)	0	4	0	—	2	—
... (2, 4)	0	4	0	—	2	—
.... (4, 5)	0	4	0	1	2	—
.. (5, 6)	0	4	0	1	2	—
(1, 2)	0	4	0	1	2	4

v	=	1	2	3	4	5	6
parent[v]	=	1	5	1	2	3	5



Pós-ordem

- Outra informação que podemos gerar a partir da execução de um DFS é a ordem em que os vértices são exauridos
 - Essa sequência também é conhecida como pós-ordem.
- Um vértice recebe a numeração de pós-ordem após examinar todas as suas arestas de saída.
- Podemos gerar essa sequência registrando o momento em que o algoritmo termina a avaliação de um vértice.

Pós-ordem

- O algoritmo de **busca em profundidade (DFS)** pode ser modificado para gerar a sequência de pós-ordem da seguinte forma:

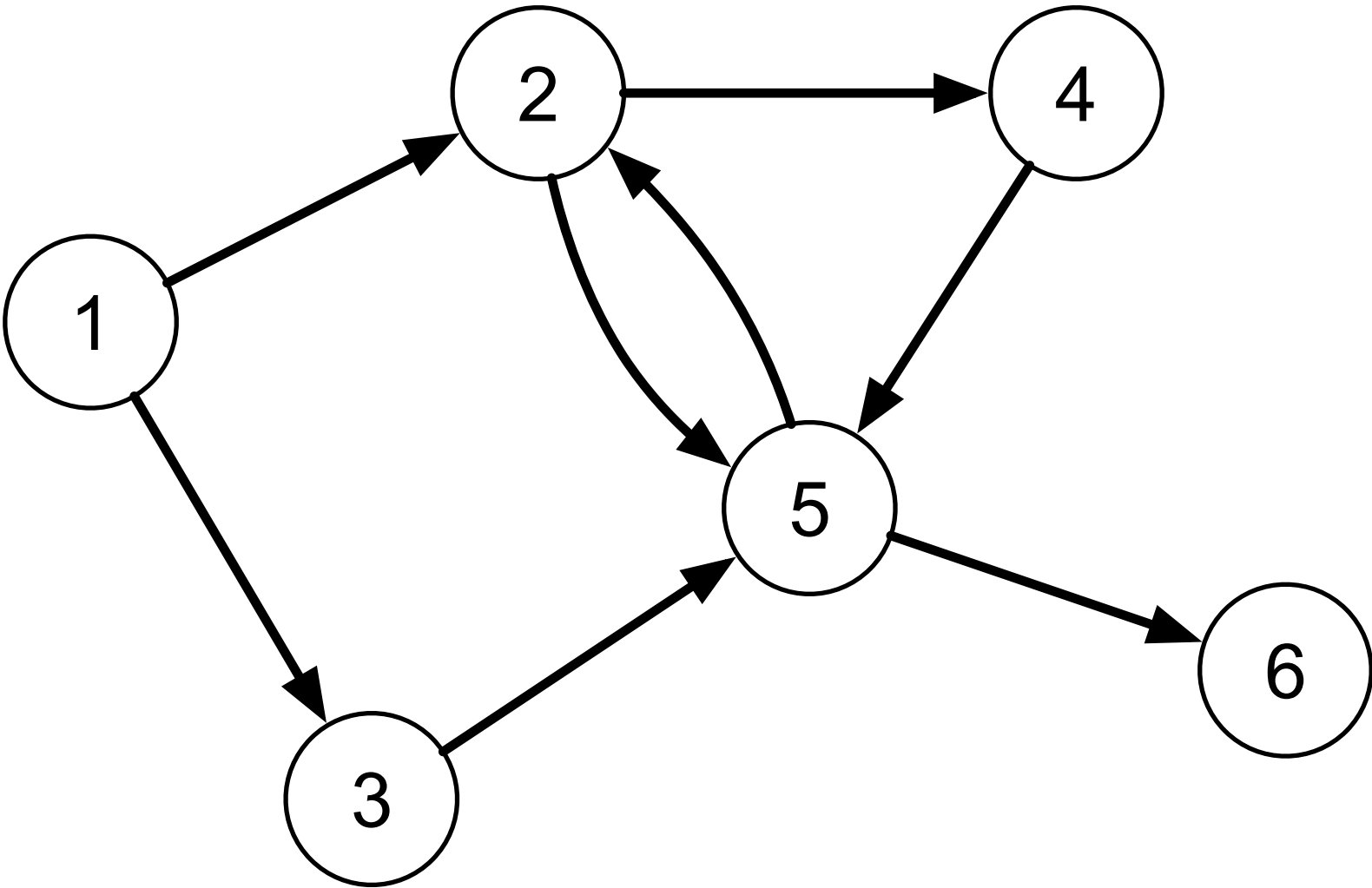
```
void dfs(int * preOrder, int * postOrder,
        int * parents) {
    int preCounter = 0;
    int postCounter = 0;
    for (vertex v=0; v < m_numVertices; v++) {
        preOrder[v] = -1;
        parents[v] = -1;
        postOrder[v] = -1;
    }
    for (vertex v=0; v < m_numVertices; v++) {
        if (preOrder[v] == -1) {
            parents[v] = v;
            dfsRecursive(
                v, preOrder, preCounter,
                postOrder, postCounter, parents);
        }
    }
}
```

```
void dfsRecursive(vertex v1, int * preOrder,
                 int & preCounter, int * postOrder,
                 int & postCounter, int * parents) {
    preOrder[v1] = preCounter++;
    EdgeNode * edge = m_edges[v1];
    while(edge) {
        vertex v2 = edge->otherVertex();
        if (preOrder[v2] == -1) {
            parents[v2] = v1;
            dfsRecursive(v2, preOrder, preCounter,
                        postOrder, postCounter, parents);
        }
        edge = edge->next();
    }
    postOrder[v1] = postCounter++;
}
```


- Como seria a execução desse algoritmo no grafo a seguir?

(1, 3)	—	—	—	—	—	—
. (3, 5)	—	—	—	—	—	—
.. (5, 2)	—	—	—	—	—	—
... (2, 5)	—	—	—	—	—	—
... (2, 4)	—	—	—	—	—	—
.... (4, 5)	—	—	—	—	—	—
.. (5, 6)	—	1	—	0	—	—
(1, 2)	—	1	4	0	3	2

v	=	1	2	3	4	5	6
postOrder[v]	=	5	1	4	0	3	2



Intervalo de vida

- O **intervalo de vida** (*lifespan*) de um vértice no contexto da busca ocorre entre o momento que ele é descoberto e o momento em que ele é exaurido
 - Definido por $preOrder[v]$ e $postOrder[v]$.

- **Importante:** o tempo de vida não pode ser definido como

$$(preOrder[v], postOrder[v])$$

pois $preOrder$ e $postOrder$ são numerações independentes!

- Observe que um vértice será sempre descoberto antes de ser exaurido.

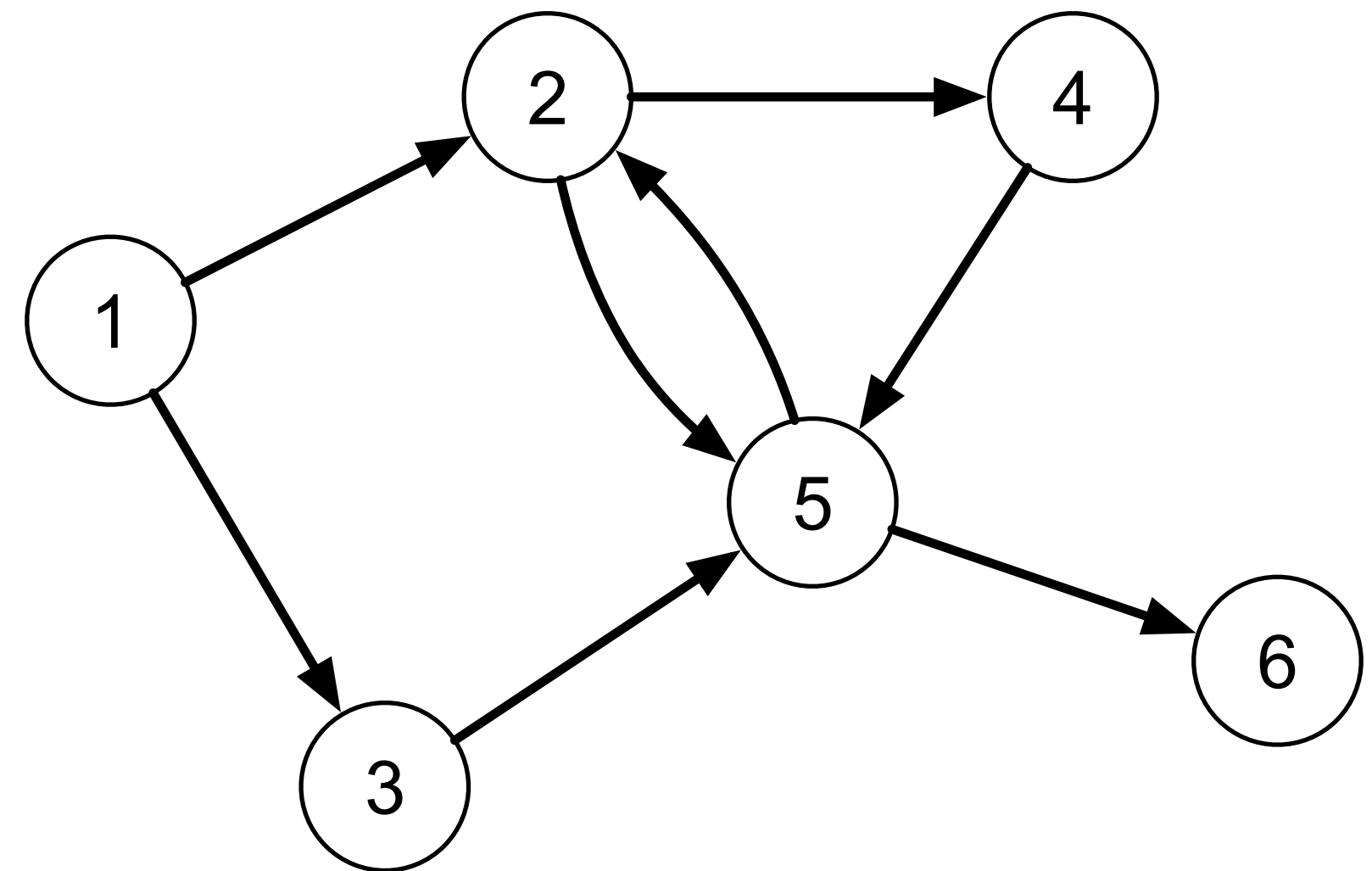
Intervalo de vida

- A ordem em que os vértices são descobertos e exauridos podem nos dizer muito sobre a relação entre eles.
- Considere dois vértices v_1 e v_2
 - Se v_1 é descoberto antes de v_2 , então v_1 é exaurido
 - Antes de v_2 ser descoberto;
 - Ou depois de v_2 ser exaurido.

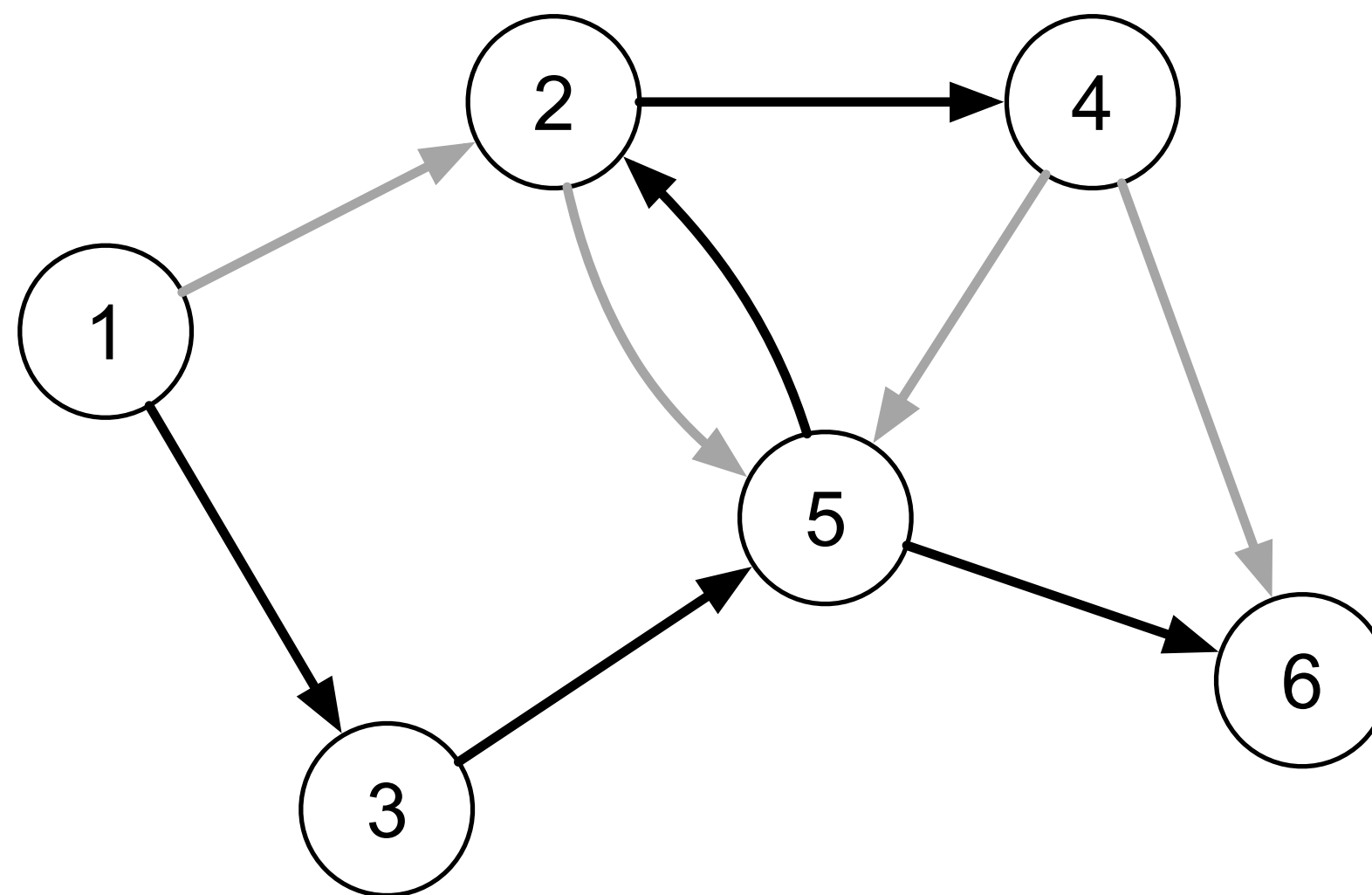
- Como podemos representar o **intervalo de vida** da execução no grafo a seguir?

(1, 3)	1
.. (3, 5)	1 3
... (5, 2)	1 3 5
.... (2, 5)	1 3 5 2
.... (2, 4)	1 3 5 2
..... (4, 5)	1 3 5 2 4
.. (5, 6)	1 3 5
... 6	1 3 5 6
(1, 2)	1

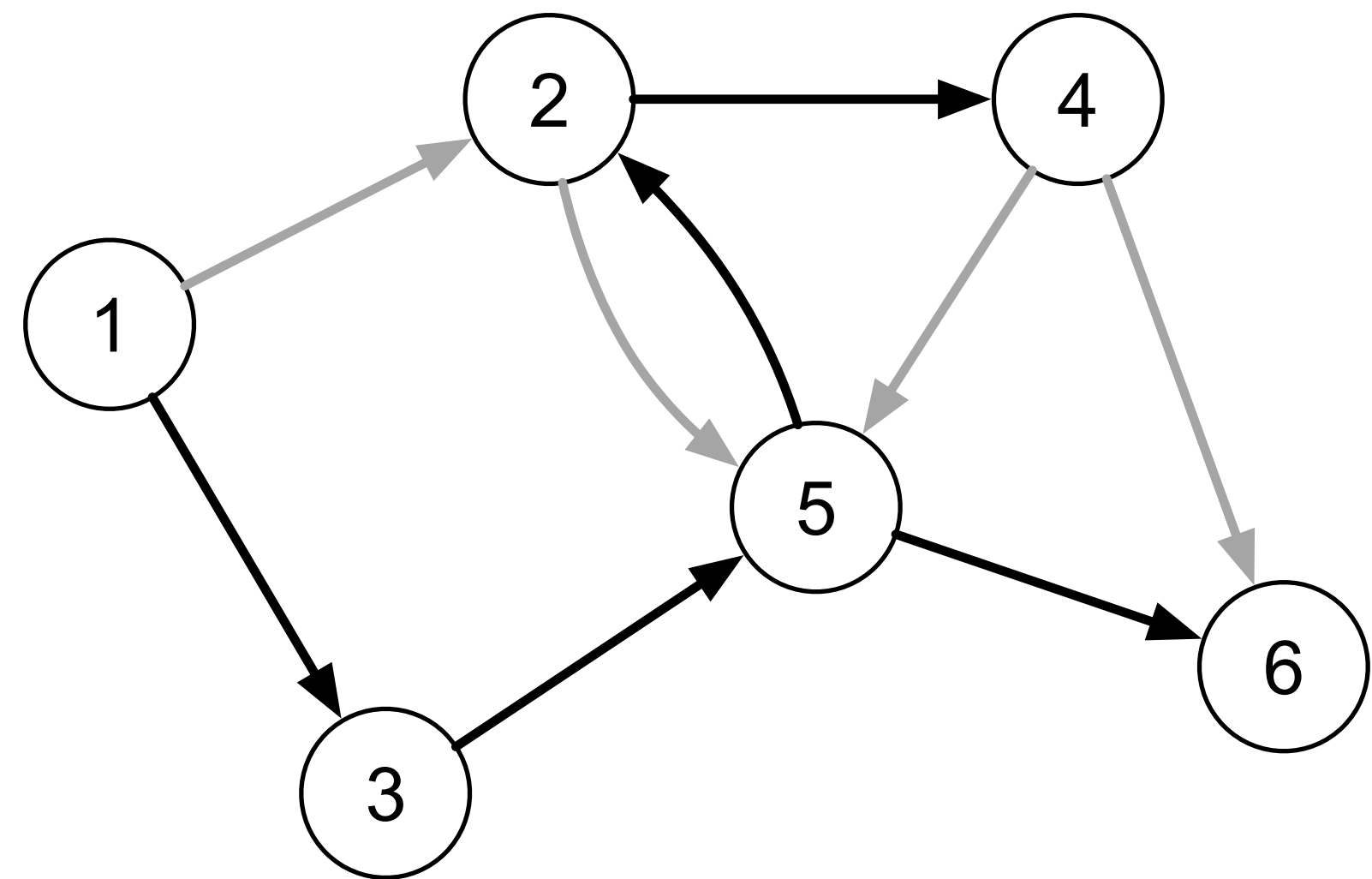
(1 (3 (5 (2 (4) 2) (6) 5) 3) 1)



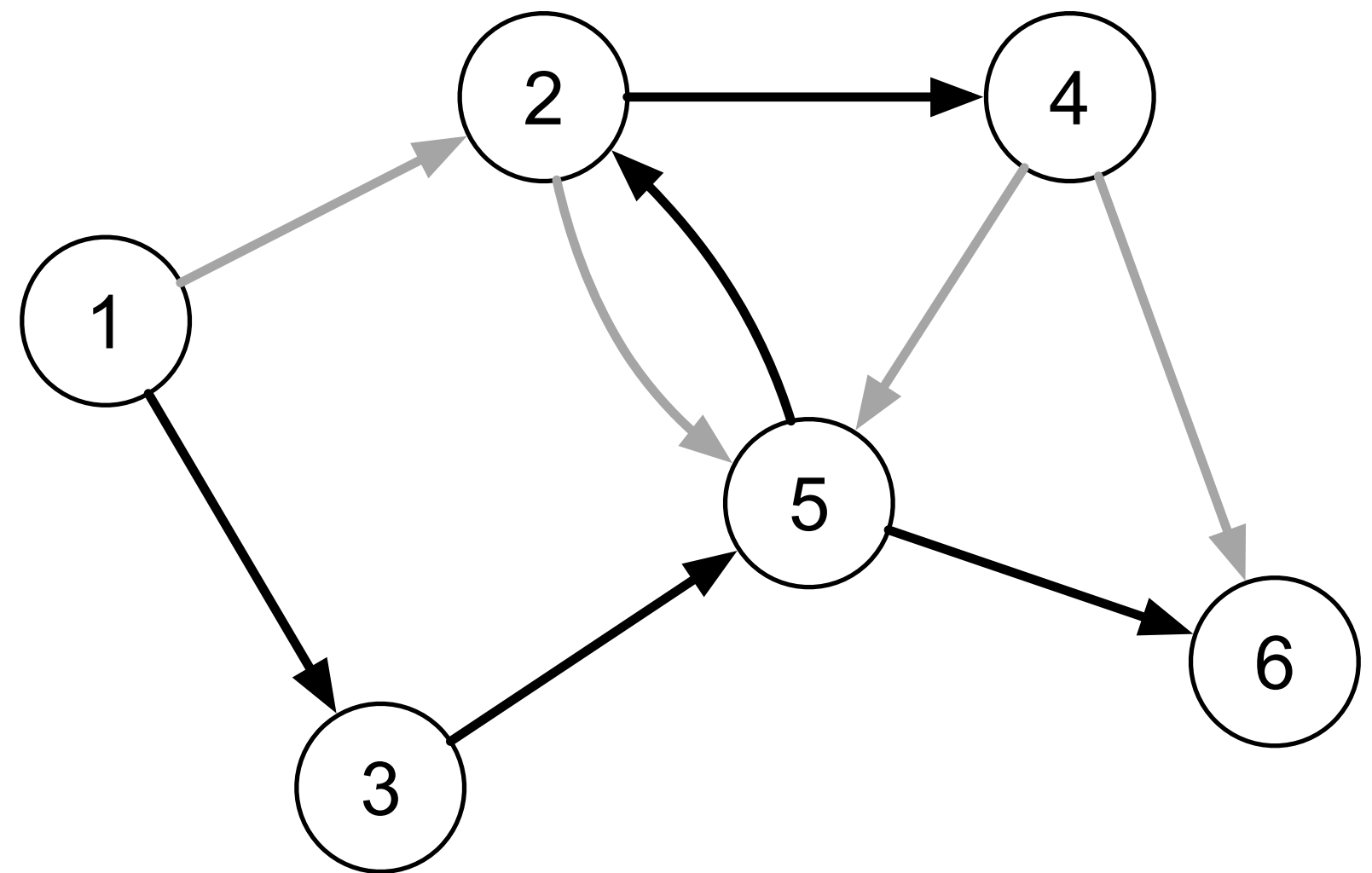
- Dados dois vértices v_1 e v_2 de uma floresta radcada produzida por uma execução DFS.
- O relacionamento desses vértices pode ser:
 - Ancestral.
 - Descendente.
 - Primo.



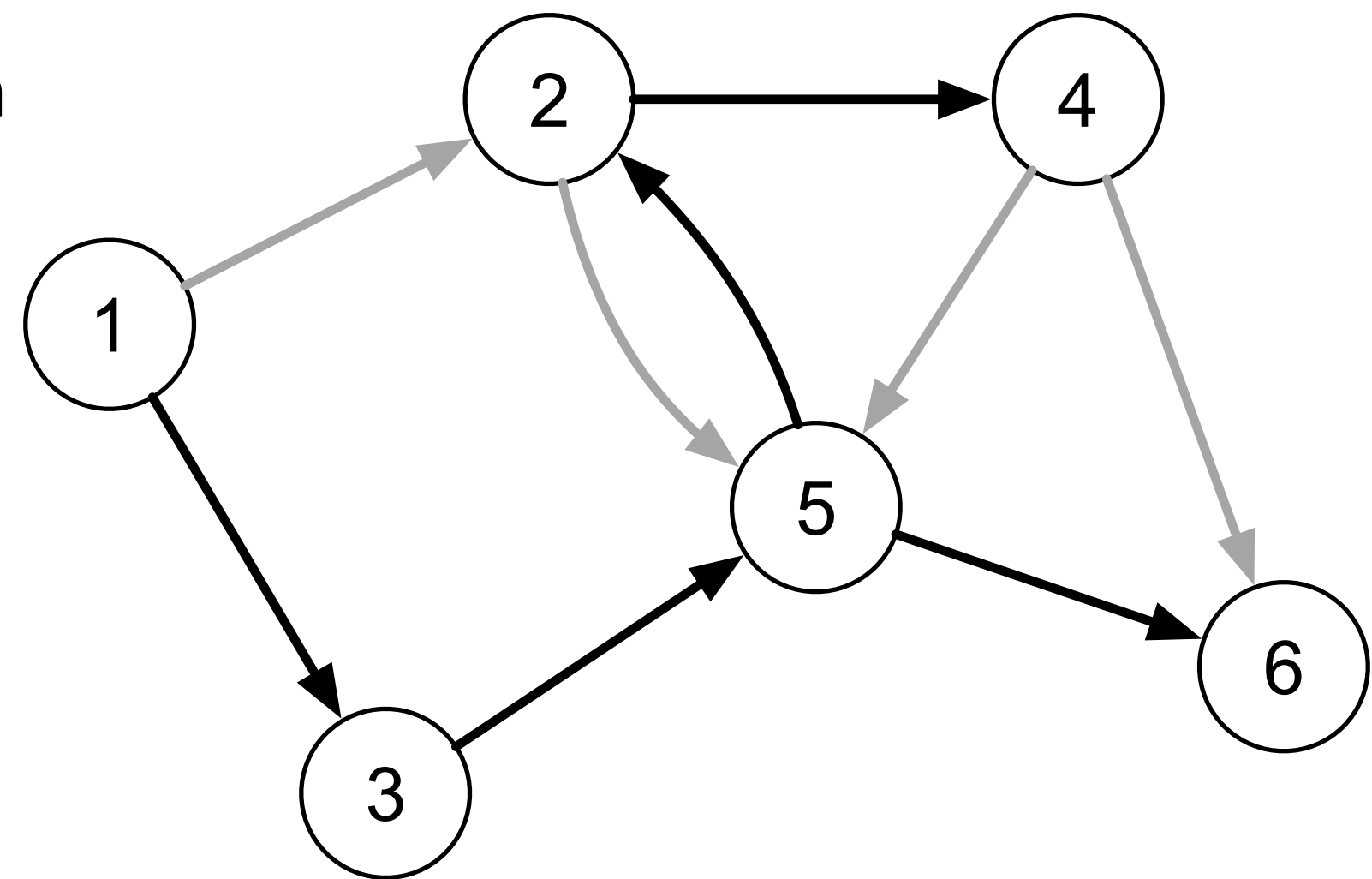
- Dados dois vértices v_1 e v_2 de uma floresta radicada produzida por uma execução DFS.
- Primos podem ser comparados:
 - v_1 é primo mais velho de v_2 se
 - $preOrder[v_1] < preOrder[v_2]$
 - v_1 é primo mais novo de v_2 se
 - $preOrder[v_1] > preOrder[v_2]$



- Arestas (v_i, v_j) que não pertencem à floresta DFS podem ser classificadas de acordo com o grau de parentesco.
- Uma aresta é de **retorno** caso v_j seja ancestral de v_i .
- Uma aresta é de **avanço** caso v_j seja descendente de v_i .
- Uma aresta é **cruzada** caso v_j seja primo de v_i .



- Arestas (v_i, v_j) que não pertencem à floresta DFS podem ser classificadas de acordo com o grau de parentesco.
- Vértices de arestas cruzadas podem estar em diferentes árvores da floresta.
- Arestas cruzadas são sempre de um primo mais novo para um primo mais velho.
- Grafos não-orientados não possuem arestas cruzadas.



- **Problema:** dada uma aresta (v_i, v_j) não pertencente à floresta DFS, como determinar algoritmicamente se:
 - É uma aresta de avanço (v_i é ancestral de v_j)
 - O intervalo de v_j **está contido** no intervalo de v_i .
 - É uma aresta de retorno (v_i é descendente de v_j)
 - O intervalo de v_j **contém** o intervalo de v_i .
 - É uma aresta cruzada (v_i é primo mais novo de v_j)
 - O intervalo de v_j **ocorre antes** do intervalo de v_i .
- Dica: utilize o intervalo de vida para gerar a definição.

- **Problema:** dada uma aresta (v_i, v_j) não pertencente à floresta DFS, como determinar algoritmicamente se:
 - É uma aresta de avanço (v_i é ancestral de v_j)
 - $preOrder[vi] < preOrder[vj]$ AND $postOrder[vi] > postOrder[vj]$
 - É uma aresta de retorno (v_i é descendente de v_j)
 - $preOrder[v_i] > preOrder[v_j]$ AND $postOrder[v_i] < postOrder[vj]$
 - É uma aresta cruzada (v_i é primo mais novo de v_j)
 - $preOrder[v_i] > preOrder[v_j]$ AND $postOrder[vi] > postOrder[vj]$
- Dica: utilize o intervalo de vida para gerar a definição.

Ciclos

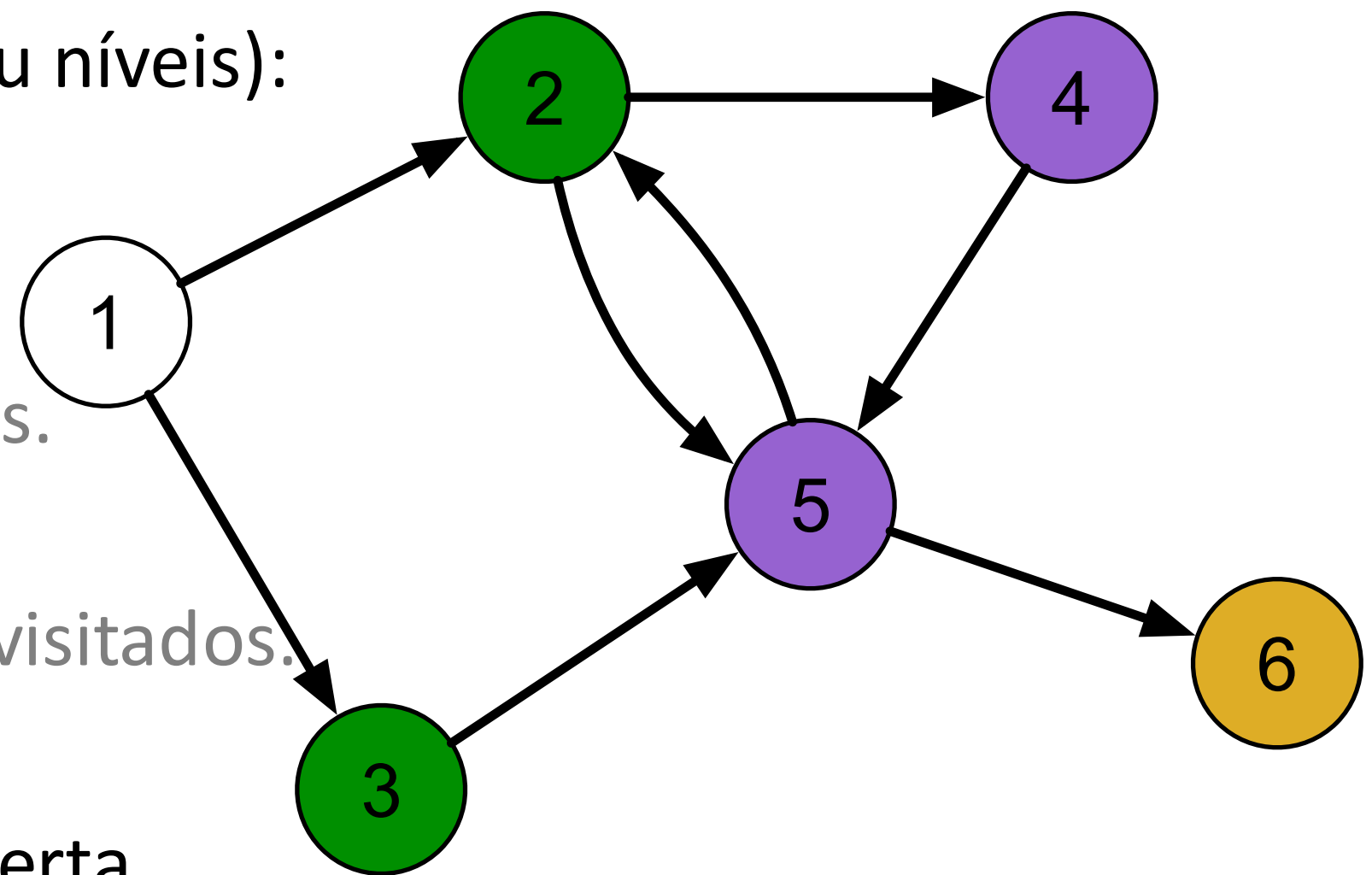
- Um **ciclo** é um caminho entre dois vértices v_i e v_j tal que $v_i = v_j$.
- Um grafo é considerado acíclico caso não possua ciclos
 - Um grafo é acíclico se e somente se possuir uma numeração topológica.
- Grafos acíclicos também são chamados de DAGs (*Direct acyclic graphs*).
- Uma floresta radicada é um DAG sem vértices com grau de entrada maior que 1.
- Uma árvore radicada é um DAG em que exatamente um vértice tem grau de entrada zero, e os demais grau de entrada 1.

- **Problema:** como determinar se um grafo $G = (V, E)$ possui ao menos um ciclo?
 - Solução ineficiente: para cada aresta (v_i, v_j) procurar um caminho de v_j até v_i .
 - Como produzir uma solução eficiente?
- Solução eficiente: execute a busca DFS e procure por uma aresta de retorno comparando os intervalos de vida encontrados para cada vértice.

```
bool hasCycle(int * preOrder, int * postOrder) {
    dfs(preOrder, postOrder);
    for (vertex v1=0; v1 < m_numVertices; v1++) {
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (preOrder[v1] > preOrder[v2]
                && postOrder[v1] < postOrder[v2]) {
                return true;
            }
            edge = edge->next();
        }
    }
    return false;
}
```

Busca em largura

- O algoritmo de **busca em largura** (BFS - *Breadth First Search*) é uma outra estratégia de varredura em um grafo.
- Ideia geral: percorrer o grafo por camadas (ou níveis):
 - Inicia visitando um vértice v_0 .
 - Visita seus vértices adjacentes.
 - Visita os vértices adjacentes dos adjacentes.
 - Que ainda não tiverem sido visitados.
 - Continua até todos os vértices terem sido visitados.
- Assim como o DFS define a ordem de descoberta dos vértices.



Busca em largura

- Uma implementação comum desse algoritmo utiliza uma fila para armazenar os vértices descobertos que ainda não foram explorados.
- Se um vértice está na fila:
 - Ele já foi numerado.
 - Seus adjacentes não foram numerados.
- Observe que essa implementação numera apenas os vértices acessíveis a partir de v_0 .

```
void bfs(vertex v0, int * order) {  
    queue<int> queue;  
    int counter = 0;  
    for (int i=0; i < m_numVertices; i++) {  
        order[i] = -1;  
    }  
    order[v0] = counter++;  
    queue.push(v0);  
    while (!queue.empty()) {  
        int v1 = queue.front();  
        queue.pop();  
        EdgeNode * edge = m_edges[v1];  
        while (edge) {  
            vertex v2 = edge->otherVertex();  
            if (order[v2] == -1) {  
                order[v2] = counter++;  
                queue.push(v2);  
            }  
            edge = edge->next();  
        }  
    }  
}
```

Busca em largura

- A seguir uma implementação que numera todos os vértices gerando uma floresta.

```
void bfsForest(int * order) {
    int counter = 0;
    for (int i=0; i < m_numVertices; i++) { order[i] = -1; }
    for (int i=0; i < m_numVertices; i++) {
        if (order[i] != -1) { continue; }
        order[i] = counter++;
        queue<int> queue;
        queue.push(i);
        while (!queue.empty()) {
            int v1 = queue.front();
            queue.pop();
            EdgeNode * edge = m_edges[v1];
            while(edge) {
                vertex v2 = edge->otherVertex();
                if (order[v2] == -1) {
                    order[v2] = counter++;
                    queue.push(v2);
                }
                edge = edge->next();
            }
        }
    }
}
```

Busca em largura

- A busca em largura a partir de um vértice cria uma árvore radcada com raiz nesse vértice inicial.
- Essa árvore pode ser representada através de um vetor de pais (*parents*).

```
void bfs(vertex v0, int * order, int * parent) {
    queue<int> queue;
    int counter = 0;
    for (int i=0; i < m_numVertices; i++) {
        order[i] = -1;
        parent[i] = -1;
    }
    order[v0] = counter++;
    parent[v0] = v0;
    queue.push(v0);
    while (!queue.empty()) {
        int v1 = queue.front();
        queue.pop();
        EdgeNode * edge = m_edges[v1];
        while(edge) {
            vertex v2 = edge->otherVertex();
            if (order[v2] == -1) {
                order[v2] = counter++;
                parent[v2] = v1;
                queue.push(v2);
            }
            edge = edge->next();
        }
    }
}
```


Busca em largura

- Analise da complexidade:
 - Cada vértice entra uma única vez na fila.
 - Todos os vértices na fila são processados uma única vez.
 - Em cada vértice v_i são verificadas $g_s(v_i)$ arestas.
- Sabendo que $\sum_{i=1}^{|V|} g_s(v_i) = |E|$, temos uma complexidade $\Theta(V + E)$ ao utilizar a estrutura de dados lista de adjacências
 - Se a estrutura de dados for matriz de adjacências a complexidade vai ser $\Theta(V^2)$.
- Se o grafo for denso, em ambas as estruturas de dados a complexidade será $\Theta(V^2)$.

Exercícios

- **Exercício:** dado um grafo $G = (V, E)$ crie um algoritmo baseado em DFS que classifica cada aresta do grafo *on-the-fly*
 - Ou seja, define se a aresta é:
 - Parte da floresta DFS.
 - De avanço.
 - De retorno.
 - Cruzada.
- O algoritmo deverá apresentar complexidade $O(V + E)$.

Exercícios

- **Exercício:** dado um grafo $G = (V, E)$ crie um algoritmo baseado em DFS que classifica cada aresta do grafo *on-the-fly*
- Solução: basta avaliar cada par de arestas (v_i, v_j) durante a varredura do grafo
 - Faz parte da floresta DFS quando v_j ainda não foi descoberto.
 - É de retorno quando v_j ainda não foi exaurido.
 - Se v_j foi exaurido:
 - É de avanço quando v_i foi descoberto antes de v_j .
 - É cruzada quando v_j foi descoberto antes de v_i .

Exercícios

- **Exercício:** dado um grafo $G = (V, E)$ crie um algoritmo baseado em DFS que classifica cada aresta do grafo *on-the-fly*.
- A definição do tipo da aresta é realizada em tempo constante apenas avaliando *endOrder* e *postOrder*.
- Logo o algoritmo é $O(V + E)$.

```
void dfsRecursive(vertex v1, int * startOrder,
                  int & startCounter, int * endOrder,
                  int & endCounter) {
    startOrder[v1] = startCounter++;
    EdgeNode * edge = m_edges[v1];
    while(edge) {
        vertex v2 = edge->otherVertex();
        if (startOrder[v2] == -1) {
            printf("(%d,%d) Tree branch\n", v1, v2);
            dfsRecursive(v2, startOrder, startCounter,
                        endOrder, endCounter);
        } else {
            if (endOrder[v2] == -1) {
                printf("(%d,%d) Return\n", v1, v2);
            } else {
                if (startOrder[v2] > startOrder[v1]) {
                    printf("(%d,%d) Forward\n", v1, v2);
                } else {
                    printf("(%d,%d) Cross\n", v1, v2);
                }
            }
        }
        edge = edge->next();
    }
    endOrder[v1] = endCounter++;
}
```

Perguntas?

`thiago.p.araujo@fgv.br`